
PROGRAMAÇÃO ORIENTADA A OBJETO (POO)

PYTHON – PILARES DA POO

PROF: HELENA

@HelenaNascimento

Bacharel em SI

2 artigos publicados

Conhecimento Avançado:

- **Análise de Dados**
- **Automação de Processos**
- **Banco de Dados**



hnasc1989



@hnasc.1989



HelenaNascimento

-
- Encapsulamento:
 - Definição
 - Prática
 - Herança:
 - Definição
 - Prática
 - Polimorfismo:
 - Definição
 - Prática
-



ENCAPSULAMENTO

Por que usar encapsulamento?

1. Segurança: Impede que o saldo de uma conta fique negativo acidentalmente.
2. Flexibilidade: Você pode mudar a lógica interna (ex: adicionar uma taxa de saque) sem quebrar o código de quem usa a classe.
3. Organização: Expõe apenas o que é necessário para o "mundo exterior".

Observação:

Em Python, o encapsulamento é baseado em **convenções de nomenclatura**, pois a linguagem não impede fisicamente o acesso aos dados como Java ou C++, mas sinaliza ao programador como ele deve interagir com os atributos.

Exemplo:

```
class ContaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.titular = titular          # Público
        self.__saldo = saldo_inicial    # Privado (Encapsulado)

    # Método para consultar o saldo (Getter)
    def consultar_saldo(self):
        return f"Saldo atual: R${self.__saldo:.2f}"

    # Método para depositar (Setter com lógica)
    def depositar(self, valor):
        if valor > 0:
            self.__saldo += valor
            print(f"Depósito de R${valor} realizado!")
        else:
            print("Valor de depósito inválido.")

    # Método para sacar (Setter com lógica)
    def sacar(self, valor):
        if 0 < valor <= self.__saldo:
            self.__saldo -= valor
            return True
        print("Saldo insuficiente ou valor inválido.")
        return False
```

```
# --- Testando o Encapsulamento ---
minha_conta = ContaBancaria("Alice", 1000)

print(minha_conta.titular)          # Acesso permitido: Alice
# print(minha_conta.__saldo)        # Erro! O Python "esconde" esse atributo

minha_conta.depositar(500)           # Forma correta de alterar o saldo
print(minha_conta.consultar_saldo()) # Forma correta de ler o saldo
```

Níveis de acesso:

Público (Public):

Sintaxe: nome_do_atributo

Protegido (Protected):

Sintaxe: _nome_do_atributo

Privado (Private):

Sintaxe: __nome_do_atributo

Getter e Setter com @property

```
class Produto:
    def __init__(self, preco):
        self.__preco = preco

    @property
    def preco(self):
        return self.__preco

    @preco.setter
    def preco(self, novo_preco):
        if novo_preco > 0:
            self.__preco = novo_preco
        else:
            print("O preço deve ser positivo!")

# Uso:
p1 = Produto(50)
p1.preco = 60 # Chama o @preco.setter automaticamente
print(p1.preco) # Chama o @property (getter)
```




HERANÇA

A **Herança** é o mecanismo que permite que uma classe (chamada de **Classe Filha** ou Subclasse) herde atributos e métodos de outra classe (chamada de **Classe Pai** ou Superclasse).

A ideia principal é o **reuso de código**. Se várias classes compartilham o mesmo comportamento, você define esse comportamento uma única vez na classe pai.

```
# Superclasse (Pai)
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def comer(self):
        print(f"{self.nome} está comendo...")

# Subclasse (Filha) - Herda de Animal
class Cachorro(Animal):
    def latir(self):
        print(f"{self.nome} diz: Au Au!")

# Subclasse (Filha) - Herda de Animal
class Gato(Animal):
    def miar(self):
        print(f"{self.nome} diz: Miau!")

# Testando
dog = Cachorro("Rex")
dog.comer() # Método herdado da classe pai
dog.latir() # Método específico da classe filha
```


[illegible]

O **Polimorfismo** é a capacidade de diferentes objetos (de classes diferentes) responderem ao **mesmo método** de formas específicas.

A palavra vem do grego: *Poli* (muitos) + *Morphos* (formas). Na prática, significa que você pode tratar objetos diferentes de maneira uniforme, sem precisar saber exatamente qual é a classe do objeto no momento da chamada.

```
class Forma:
    def area(self):
        pass # Método genérico

class Quadrado(Forma):
    def __init__(self, lado):
        self.lado = lado
    def area(self):
        return self.lado ** 2

class Circulo(Forma):
    def __init__(self, raio):
        self.raio = raio
    def area(self):
        return 3.14 * (self.raio ** 2)

# O Polimorfismo em ação:
formas = [Quadrado(4), Circulo(5), Quadrado(10)]

for f in formas:
    # Não importa se é quadrado ou círculo, chamamos o mesmo método .area()
    print(f"Área: {f.area()}")
```
